

EdTech LMS Expansion

Technical & Product Implementation Specification — From Standard LMS to Expanded Global Learning Marketplace

Purpose: Convert the existing standard EdTech LMS into an expanded learning marketplace, modular learning engine, creator platform, education CRM, and commerce platform. This document consolidates the supplied product specification and the implementation decisions discussed for homepage, roles, storage, video delivery, learning progress, webinars, dashboards, and phased development.

Source basis: the supplied 2,451-line product specification, the current LMS platform references, and the implementation discussion. Source-derived requirements are preserved; implementation decisions added during planning are identified by their technical nature rather than presented as claims about the source.

1. Executive Summary

The target product is not simply another LMS. It is a Global Learning Marketplace + Modular Learning Engine + Creator Economy + Education CRM, with a central commercial innovation: students can learn only what they need, buy individual topics, accumulate eligible topic purchases as course credit, and later upgrade to the complete course without losing eligible credit.

The platform contains four major product areas:

- Learning Marketplace — students discover courses, topics, instructors, learning paths, webinars and other learning products.
- Creator Platform — instructors create courses, modules, topics, lessons and resources; set pricing; publish; manage students; view reviews, analytics, earnings and payouts.
- Education CRM — a unified student record connects leads, searches, views, purchases, learning progress, questions, certificates, payments and communications.
- Payment + Entitlement Engine — orders, payments, refunds, entitlements, topic credits, course upgrades, instructor earnings and payouts.

The central product loop is:

Discover Topic → Buy Topic → Entitlement → Learn → Buy More Topics → Accumulate Eligible Credit → Upgrade to Full Course → Complete → Certificate → Career / Next Learning Path

2. Product Vision and Differentiator

2.1 Positioning

The platform should not be positioned simply as an 'online course platform' or as another Udemy. The intended positioning is a global learning marketplace where students can buy exactly the skills they need and progressively upgrade those purchases into complete courses.

2.2 Atomic learning commerce

The unit of learning is smaller and more flexible than a complete course. A course is decomposed into modules, topics and lessons. A student can purchase an individual topic, learn it, purchase additional topics, and later apply eligible historical topic purchases as credit toward the current full-course price under defined business rules.

2.3 Extended learning ecosystem

- Topic
- Module
- Course
- Learning Path
- Certificate
- Career

Horizontally, the platform can support recorded learning, live learning, cohorts, workshops, mentorship, projects, community and certification.

3. Product Scope

3.1 Major portals

Portal	Primary users	Purpose
Public / Marketplace	Visitors, students	Discovery, search, course/topic/instructor/webinar pages, registration and conversion
Student	Students	Learning, purchases, progress, certificates, orders, wishlist, messages and profile
Instructor / Creator	Instructors	Course creation, curriculum, topics, pricing, students, reviews, live sessions, earnings, payouts and analytics
Admin	Operations team	Users, instructors, courses, topics, orders, refunds, payments, reviews, reports, support and marketing
Super Admin	Business/platform owners	GMV, revenue, take rate, students, creators, countries, currencies, taxes, growth, fraud and infrastructure

3.2 Roles

- STUDENT
- INSTRUCTOR

- ADMIN
- SUPER_ADMIN

Use one authentication system with role-based access control (RBAC), rather than independent authentication systems for every dashboard.

4. Public Website and Homepage

4.1 Homepage goals

From the user perspective, the homepage must answer: What can I learn? Why should I learn here? How do I start?

4.2 Homepage sections

1. Hero section with the core value proposition and prominent search.
2. Search: 'What do you want to learn?'
3. Categories such as Software Development, AI & Machine Learning, Data Science, Cloud, Cybersecurity, Business and Design.
4. Trending Topics.
5. Popular Courses.
6. Learning Paths / career paths.
7. Upcoming Webinars / Live Sessions.
8. Top Instructors.
9. Free Topics.
10. New Courses.
11. Recommended for You (when personalization data exists).
12. Why this platform / topic-credit differentiator.
13. Career / placement outcome section.
14. Become an Instructor section.
15. Student testimonials / reviews.
16. Footer with platform, student, instructor and company links.

4.3 Public routes

```

/
/explorer
/courses
/topics
/course/:slug

```

/topic/:slug
/instructors
/instructor/:slug
/webinars
/webinar/:slug
/learning-paths
/about
/contact

4.4 Course card content

- Course image/thumbnail
- Title
- Instructor
- Rating and review count
- Student count
- Number of modules/topics
- Duration
- Level
- Language
- Starting topic price
- Full course price
- View Course CTA

4.5 Topic card content

- Topic title
- Course
- Instructor
- Duration
- Difficulty
- Skills
- Rating
- Topic price
- Free/paid state
- View Topic CTA

4.6 Homepage differentiator section

Show the atomic-learning model clearly: 'Learn only what you need. Buy individual topics. Your eligible topic purchases become course credit. Upgrade later without losing eligible credit.'

5. Authentication, Registration and RBAC

5.1 Common authentication

- Email/password login
- Registration
- Password reset
- Email verification
- Optional social login/OAuth
- Session/refresh-token handling
- Role retrieval after authentication

5.2 Redirects

Role	Dashboard
STUDENT	/dashboard
INSTRUCTOR	/instructor/dashboard
ADMIN	/admin/dashboard
SUPER_ADMIN	/super-admin/dashboard

5.3 Authorization rules

- Students can access only their own private learning, order and profile data.
- Instructors can modify only courses/content they own or are explicitly authorized to manage.
- Admins can perform operational actions granted to the admin role.
- Super Admin has business/platform control.
- Backend authorization is mandatory; frontend route guards are not sufficient.

6. Student Portal

6.1 Student profile fields

Group	Fields
Identity	First name, last name, email, phone, profile photo
Location	Country, state, city, timezone
Preferences	Preferred language, learning preferences
Education	Qualification, institution, graduation year
Skills	Skills / interests

Learning	Enrolled courses, purchased topics, progress, quiz attempts, certificates
Commerce	Orders, payments, entitlements, topic credits, upgrades
Engagement	Wishlist, messages, questions, reviews, notifications

6.2 Student routes

/dashboard
 /dashboard/courses
 /dashboard/topics
 /dashboard/learning
 /dashboard/orders
 /dashboard/certificates
 /dashboard/wishlist
 /dashboard/messages
 /dashboard/profile

6.3 Student dashboard

- Continue Learning
- Overall course progress
- Module/topic/lesson completion
- Purchased topics
- Course-credit/upgrade opportunities
- Upcoming webinars
- Certificates
- Recent orders
- Wishlist
- Notifications/messages

7. Instructor / Creator Portal

7.1 Instructor profile fields

Group	Fields
Personal	Name, email, phone, profile photo, country
Professional	Bio, expertise, skills, experience, qualifications, current organization
Verification	Identity status, verification status, payout/KYC status
Marketplace	Courses, topics, students, reviews, earnings, analytics
Payout	Payment account/provider onboarding state, payout status

7.2 Instructor routes

/instructor/dashboard
/instructor/courses
/instructor/course/new
/instructor/course/:id
/instructor/curriculum
/instructor/topics
/instructor/pricing
/instructor/students
/instructor/reviews
/instructor/questions
/instructor/live
/instructor/earnings
/instructor/payouts
/instructor/analytics

7.3 Course creation workflow

Create Course → Create Module → Create Topic → Add Lessons → Upload Video/Resources → Set Topic Price → Set Full Course Price → Preview → Submit for Review → Publish

The creator experience should remain simple even though the underlying platform is complex.

8. Admin Portal

8.1 Admin routes

/admin/dashboard
/admin/users
/admin/students
/admin/instructors
/admin/courses
/admin/topics
/admin/webinars
/admin/orders
/admin/payments
/admin/refunds
/admin/reviews
/admin/reports
/admin/support
/admin/marketing

8.2 Admin functions

- Manage students/users
- Review and approve instructors
- Review, approve/reject and moderate courses
- Manage topics
- Manage webinars
- Manage categories
- Review orders and payments

- Process/refine refund workflows
- Moderate reviews/questions
- Support management
- Marketing operations
- Operational reports

8.3 Content moderation

Instructor-submitted courses should move through automated checks and admin review before publication. Checks can include copyright, quality, audio/video quality, spam, misleading claims, dangerous content, plagiarism and applicable AI-generated-content disclosure requirements.

9. Super Admin Portal

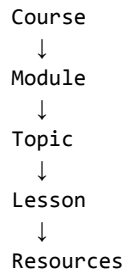
9.1 Business control center

- GMV
- Revenue
- Take Rate
- Students
- Creators
- Courses
- Topics
- Orders
- Refunds
- Payouts
- Countries
- Currencies
- Taxes
- Growth
- Infrastructure
- Fraud
- Support

Super Admin should be separated from routine admin operations. It is the business/platform control layer.

10. Course Content Architecture

10.1 Hierarchy



10.2 Course fields

Field	Purpose
courseId	Unique identifier
title / slug	Course identity and URL
description / shortDescription	Course presentation
thumbnail / banner	Marketplace visuals
instructorId	Course owner
category / subcategory	Discovery taxonomy
level	Beginner/intermediate/advanced etc.
language	Course language
skills	Search/recommendation metadata
requirements	Prerequisites
learningObjectives	Expected outcomes
coursePrice / currency	Full-course commercial price
status / visibility	Draft/review/published/archived and visibility
publishedAt	Publication timestamp

10.3 Module fields

- moduleId
- courseId
- title
- description
- order
- status

10.4 Topic fields

- topicId
- moduleId
- title
- description

- price
- duration
- difficulty
- skills
- prerequisites
- learningObjectives
- isFree
- status
- order

10.5 Lesson fields

- lessonId
- topicId
- title
- videoId
- playback reference
- duration
- transcript
- resources
- order
- status

11. Video Storage and Delivery Architecture

11.1 Storage principle

Do not store large video binaries inside MongoDB. Store video files in object storage such as AWS S3. MongoDB stores video metadata, processing state and references.

11.2 Upload flow

Instructor → Frontend → Backend / Upload authorization → S3 original upload → Video processing → Multiple renditions → HLS manifest → CDN delivery → Student player

11.3 S3 organization

```
courses/{courseId}/modules/{moduleId}/topics/{topicId}/lessons/{lessonId}/
  source/
  hls/
```

```

1080p/
720p/
480p/
360p/
thumbnails/
captions/
transcripts/

```

11.4 MongoDB video metadata

```

{
  lessonId,
  videoId,
  sourceObjectKey,
  processingStatus,
  duration,
  thumbnailObjectKey,
  hlsManifestObjectKey,
  availableQualities: [1080, 720, 480, 360],
  captions,
  transcript,
  createdAt
}

```

11.5 Resolution handling

The uploaded source is processed into multiple renditions. HLS adaptive bitrate streaming lets the player select an appropriate quality based on network/device conditions. A player may expose Auto plus available quality options.

11.6 Video processing stages

```

V1: Secure S3 video delivery
V2: MediaConvert + HLS
V3: CloudFront signed URLs/cookies
V4: Watermarking
V5: Advanced anti-piracy

```

Do not build custom DRM or advanced anti-piracy as the first milestone. Use mature managed services and add protection progressively.

11.7 Security

- Never expose permanent unrestricted video URLs.
- Authorize access against course/topic entitlement.
- Use signed access for private playback.
- Verify the user owns the required course/topic entitlement before issuing playback access.
- Record access/audit events where appropriate.

12. Learning Progress and Completion Tracking

12.1 Progress hierarchy

Student → Course → Module → Topic → Lesson → Video Progress

12.2 Learning progress record

```
{
  userId,
  courseId,
  moduleId,
  topicId,
  lessonId,
  watchedSeconds,
  totalSeconds,
  progressPercent,
  completed,
  lastWatchedAt
}
```

12.3 Video progress

During playback, the frontend periodically sends the current playback position to the backend. The backend stores watched time, total duration, percentage, completion state and last-watched timestamp. When the learner returns, the player can resume from the saved position.

12.4 Completion

A lesson can be marked complete after the configured completion threshold is satisfied. Topic/module/course completion can be derived from the completion state of their required child lessons/topics.

12.5 Dashboard example

```
React Masterclass
Progress: 44%
Modules: 2 / 5
Topics: 11 / 24
Lessons: 38 / 86
```

12.6 Completion event flow

COURSE_COMPLETED → Issue Certificate → Request Review → Recommend Next Course / Learning Path

13. Webinar Architecture

13.1 Webinar is a separate product/content type

Webinars should not be forced into the course-module hierarchy. They have their own scheduling, registration, delivery, attendance and recording lifecycle.

13.2 Webinar fields

Group	Fields
Basic	webinarId, title, description, thumbnail, instructorId, category
Schedule	startTime, endTime, timezone, capacity
Delivery	meetingProvider, meetingUrl, recordingUrl
Commerce	price, currency, paymentRequired
Status	Draft, Scheduled, Live, Completed, Cancelled
Engagement	registrations, attendance, questions, reviews
Resources	slides, PDFs, recordings, supporting files

13.3 Webinar storage

MongoDB → webinar metadata, schedule, instructor, registration and attendance references
S3 → thumbnail, recording, slides, PDFs and other resources

13.4 Live delivery

Use a managed live-video provider initially. Store the provider session/meeting identifier and protected access reference in the application database. After the event, the recording can be stored in object storage and attached to the webinar record.

14. Learning Paths

Learning paths connect individual topics/courses to a larger skill or career journey.

```
AI Engineer Path
  Python
  ↓
  Statistics
  ↓
  Machine Learning
  ↓
  Deep Learning
  ↓
  LLMs
  ↓
  RAG
  ↓
  AI Agents
  ↓
  Capstone
  ↓
  Certificate
```

A learning path can contain topics from different courses. This is a major benefit of the atomic topic model.

15. Marketplace and Search

15.1 Discovery

- Courses
- Topics
- Instructors
- Skills
- Categories
- Learning Paths
- Webinars
- Free Topics

- New/Trending content
- Recommendations

15.2 Search behavior

Search should eventually return courses, topics, instructors and learning paths rather than only matching course titles. A managed search engine or search index can sit alongside MongoDB.

15.3 Topic taxonomy

Category → Skill → Subject → Topic

Example: Software Development → Backend → Node.js → Authentication → JWT Refresh Tokens.

15.4 Topic metadata

- Title
- Description
- Skills
- Difficulty
- Duration
- Prerequisites
- Learning objectives
- Language
- Instructor
- Rating
- Price
- Course
- Module

16. Cart, Payments, Entitlements and Topic Credit

16.1 Commerce flow

Cart → Checkout → Payment → Order → Entitlement → Learning Access

16.2 Entitlement

Entitlement is the authoritative record of what a student is allowed to access. Do not infer access from frontend state.

Topic purchase → TOPIC entitlement
Course purchase → COURSE entitlement

16.3 Access decision

```

Does topic require purchase?
↓
Is course owned? YES → Allow
↓ NO
Is topic owned? YES → Allow
↓ NO
Is topic free? YES → Allow
↓ NO
403 / Purchase required

```

16.4 Topic credit example

Course price:	\$100
Topic A:	\$10
Topic B:	\$15
Eligible credit:	-\$25
Upgrade price:	\$75

The exact eligibility rules must be explicitly defined before implementation, including coupons, refunds, bundles, promotions, currency conversion, taxes, partial refunds and duplicate purchases.

16.5 Pricing architecture

```

Product → Currency → Base Price → Regional Pricing → Tax → Promotion → Coupon → Bundle Discount
→ Eligible Credit → Final Price

```

16.6 Refund flow

```

Purchase → Refund eligibility → Refund → Entitlement adjustment/revocation → Credit adjustment →
Instructor balance adjustment → Ledger update

```

17. Financial Ledger and Instructor Economics

17.1 Ledger

Financial history should not be reconstructed from orders alone. Use a proper ledger.

- ORDER_PAYMENT
- PLATFORM_FEE
- INSTRUCTOR_EARNING
- TAX
- REFUND
- CHARGEBACK
- PAYOUT
- ADJUSTMENT
- COURSE_CREDIT

17.2 Instructor earnings

Gross sales → Discounts → Refunds → Payment fees → Platform commission → Tax adjustments → Instructor earnings

17.3 Instructor dashboard metrics

- Gross revenue
- Students
- Topics sold
- Courses sold
- Conversion
- Available balance
- Pending balance
- Payout history

18. CRM and Event Architecture

18.1 Unified student record

- Lead information
- Name
- Email
- Phone
- Country
- Source
- Campaign
- Interest
- Student profile
- Courses
- Topics
- Purchases
- Learning progress
- Last activity
- Engagement

- Pipeline

18.2 Pipeline

Visitor → Lead → Registered → Topic Interest → Topic Buyer → Multi-topic Buyer → Course Upgrade
→ Completed → Returning Student

18.3 Event architecture

- COURSE_VIEWED
- TOPIC_VIEWED
- TOPIC_PREVIEW_STARTED
- TOPIC_PREVIEW_COMPLETED
- TOPIC_ADDED_TO_CART
- CHECKOUT_STARTED
- PAYMENT_FAILED
- PAYMENT_SUCCESS
- TOPIC_PURCHASED
- COURSE_PURCHASED
- COURSE_UPGRADED
- VIDEO_STARTED
- VIDEO_COMPLETED
- TOPIC_COMPLETED
- QUIZ_COMPLETED
- COURSE_COMPLETED
- CERTIFICATE_ISSUED

18.4 Automation examples

WHEN student owns 3+ topics AND full course not owned → show/send course-upgrade offer
WHEN checkout abandoned → wait defined period → send reminder
WHEN course completed → certificate + review request + next-course recommendation
WHEN inactive for 14 days → re-engagement campaign

19. Communication

- Email
- WhatsApp
- SMS

- Push
- Telegram

Channels should be introduced progressively. Communication records should be associated with the student and relevant event/campaign.

20. Certificates

20.1 Certificate data

- Certificate ID
- Student
- Course
- Instructor
- Completion percentage
- Issue date
- Verification URL

20.2 Verification

`platform.com/verify/CERT-928381`

Certificates should be verifiable online rather than treated only as downloadable PDFs. Later they can connect to skills, portfolios, LinkedIn and career profiles.

21. Community, Mentorship and Live Learning

- Course community
- Topic discussions
- Instructor Q&A
- Student groups
- Study groups
- Announcements
- Live course
- Live workshop
- Cohort
- 1:1 mentorship

- Office hours

These are expansion features after the core marketplace, learning and commerce loop is stable.

22. AI Features — Later Phase

22.1 AI Student Tutor

Student question → Retrieve course/topic content → RAG → Context-grounded answer

22.2 AI Learning Assistant

- Use progress, quiz scores and weak areas
- Recommend next topic
- Generate practice
- Explain difficult content
- Summarize topic content

22.3 AI Instructor Assistant

Instructor uploads video/PDF/slides; AI can later assist with transcript, summary, quiz questions, flashcards, chapter markers, descriptions and SEO metadata.

22.4 AI CRM

- Lead scoring
- Next-best action
- Student re-engagement
- Course/topic recommendation
- Sales/admission assistance

AI should not be the first implementation milestone. Build the underlying content, commerce, progress and event data first.

23. Database Architecture

23.1 Core collections / entities

```
users
instructor_profiles
courses
modules
topics
lessons
resources
orders
order_items
```

payments
 refunds
 entitlements
 topic_credits
 course_upgrades
 learning_progress
 video_progress
 quiz_attempts
 reviews
 ratings
 questions
 answers
 wishlists
 cart
 certificates
 live_sessions
 bookings
 leads
 lead_events
 campaigns
 communication_logs
 notifications
 coupons
 promotions
 payouts
 instructor_earnings
 audit_logs

23.2 Relationship model

```

graph TD
    Course --> Module
    Module --> Topic
    Topic --> Lesson
    Lesson --> Resource
    User --> Order
    Order --> OrderItem[Order Item]
    OrderItem --> Entitlement
    Entitlement --> TopicCourse[Topic / Course]
  
```

23.3 Storage separation

Data	Storage
Users, roles, profiles	MongoDB
Course/module/topic/lesson metadata	MongoDB
Orders/payments/entitlements	MongoDB
Progress/quiz/certificate metadata	MongoDB
CRM/events/notifications metadata	MongoDB
Video source and renditions	Object storage such as S3

PDFs/slides/resources/images	Object storage such as S3
Video CDN delivery	CloudFront or equivalent CDN
Cache/session/high-speed transient data	Redis

24. Globalization

24.1 Internationalization

Design UI strings for i18n from the beginning rather than hardcoding English into React.

- English
- Spanish
- French
- German
- Portuguese
- Arabic
- Hindi
- Additional languages later

24.2 Currency

Store price and currency separately. Do not permanently replace database prices with converted values.

```
price
currency
displayPrice
exchangeRate
```

24.3 Time zones

Store live-event timestamps in UTC and convert to the student's local timezone in the frontend.

24.4 Content language

- Course language
- Subtitle language
- Audio language
- Transcript language

25. Video, File and Content Storage Policy

- MongoDB stores metadata and references, not large binary videos.

- Object storage stores original uploads, processed video renditions, thumbnails, captions and downloadable resources.
- Private content is accessed through controlled/signed delivery.
- Every uploaded asset should have ownership information and a relation to its parent course/module/topic/lesson/webinar.
- File processing should be asynchronous so a large upload does not block the main request.
- Processing status should be visible to the instructor: Uploading → Processing → Ready / Failed.
- Deleted or replaced content should have lifecycle rules so storage does not grow indefinitely.
- Backups and retention policies should be defined before production.

26. Security

- JWT + refresh-token approach initially, with secure token handling.
- RBAC for STUDENT, INSTRUCTOR, ADMIN and SUPER_ADMIN.
- Instructor ownership checks on every course/content modification.
- Backend authorization for all private content.
- Signed video access rather than permanent unrestricted URLs.
- Payment status must be verified server-side.
- Payment webhook signatures must be verified.
- Sensitive admin operations must be audited.
- Audit log should record who, what, when, IP/resource where appropriate, old value and new value.
- Protect against manipulated prices, replayed webhooks, unauthorized video access and cross-instructor access.

27. Accessibility

- WCAG-oriented UI
- Keyboard navigation
- Captions
- Transcripts
- Screen-reader labels
- Adequate contrast
- Accessible playback controls

28. Legal, Compliance and Marketplace Operations

Before global launch, the platform needs appropriate professional review for legal and tax requirements.

- Terms of Service
- Privacy Policy
- Cookie Policy
- Refund Policy
- Creator Agreement
- Marketplace Agreement
- Copyright Policy
- DMCA process where applicable
- Community Guidelines
- Acceptable Use Policy
- Data retention policy
- GDPR assessment where applicable
- India DPDP assessment where applicable
- CCPA/CPRA assessment where applicable
- VAT/GST/sales-tax assessment
- Creator tax reporting requirements
- Consumer-protection requirements

Global tax and marketplace structure should determine the final payment architecture. Do not treat tax, seller-of-record, creator payout and compliance rules as frontend features.

29. Instructor Verification and Content Moderation

- Instructor identity verification
- Country verification
- Tax information
- Payment account
- Bank/payment-provider onboarding
- Course quality review
- Copyright review

- Spam/misleading-content checks
- Plagiarism checks
- Applicable AI-content review

Use payment-provider KYC/onboarding where possible rather than building an independent identity-verification system.

30. Recommendation Engine

30.1 Progressive implementation

- V1: Rules
- V2: Collaborative filtering
- V3: ML recommendation system
- V4: AI personalized learning path

30.2 Example

User buys React topic → recommend related React topics → observe behavior → personalize later

31. Technical Architecture

31.1 Initial architecture

Layer	Recommended direction
Frontend	React + TypeScript + Tailwind
Backend	FastAPI + Python
Database	MongoDB Atlas initially, as proposed in the supplied specification
Cache	Redis
Object storage	AWS S3
CDN	CloudFront
Search	OpenSearch / Algolia / similar managed search
Payments	Stripe Connect plus appropriate regional/local provider based on business setup
Email	AWS SES / transactional provider
Video	S3 + MediaConvert + CloudFront
Live	LiveKit / managed provider
Queue	AWS SQS
Monitoring	Sentry + CloudWatch
Authentication	JWT initially + OAuth/social login
Deployment	Docker, AWS ECS/Fargate, ALB, Route53, ACM
CI/CD	GitHub Actions

31.2 Modular monolith first

Do not begin with separate microservices for course, payment, video, CRM, user, notification and analytics. Start with a modular monolith, keep domain boundaries clean, and extract services later when scale justifies it.

31.3 Backend modules

```
backend/  
  app/  
    main.py  
    auth/  
    users/  
    instructors/  
    courses/  
    modules/  
    topics/  
    lessons/  
    catalog/  
    search/  
    cart/  
    orders/  
    payments/  
    entitlements/  
    credits/  
    refunds/  
    learning/  
    progress/  
    quizzes/  
    certificates/  
    reviews/  
    discussions/  
    crm/  
    campaigns/  
    notifications/  
    live/  
    mentorship/  
    analytics/  
    admin/  
    common/
```

31.4 API structure

```
/api/v1/auth  
/api/v1/users  
/api/v1/courses  
/api/v1/modules  
/api/v1/topics  
/api/v1/catalog  
/api/v1/search  
/api/v1/cart  
/api/v1/orders  
/api/v1/payments  
/api/v1/entitlements  
/api/v1/credits  
/api/v1/upgrades  
/api/v1/learning  
/api/v1/progress  
/api/v1/reviews  
/api/v1/discussions  
/api/v1/instructor  
/api/v1/crm  
/api/v1/admin
```

32. Testing Strategy

32.1 Unit tests

- Pricing
- Credits
- Tax
- Entitlement
- Refund
- Commission

32.2 Integration tests

- Payment → Order → Entitlement
- Refund → Entitlement revocation
- Upgrade → Credit

32.3 API tests

Test every endpoint, authorization rule and validation condition.

32.4 E2E flow

Register → Browse → Buy Topic → Watch → Buy Second Topic → Upgrade Course → Complete → Certificate

32.5 Security tests

- Student attempting instructor endpoint
- Instructor A attempting Instructor B's course
- Unauthenticated access to private video
- Expired entitlement attempting playback
- Manipulated checkout price
- Replayed payment webhook

32.6 Load testing

Scale testing should increase progressively. Do not optimize for extreme concurrency before the product has real usage.

33. Analytics

33.1 Marketplace metrics

- Visitors

- Course views
- Topic views
- Add-to-cart
- Checkout starts
- Purchases

33.2 Learning metrics

- Video starts
- Video completion
- Topic completion
- Quiz scores
- Course completion

33.3 Commercial metrics

- GMV
- Revenue
- AOV
- Topic revenue
- Course revenue
- Upgrade rate
- Refund rate
- Creator revenue

33.4 Core differentiator KPI

Topic → Course Conversion Rate: the percentage of students who purchase topics and later purchase the complete course.

33.5 Additional metrics

- Student activation
- Topic purchase rate
- Course purchase rate
- Completion rate
- 7/30/90-day retention
- Published courses

- Published topics
- Topic sales
- Course sales
- Creator revenue
- Rating
- Marketplace GMV
- Take rate
- CAC
- LTV
- Refund rate
- Conversion rate

34. Marketplace Cold Start

The marketplace has a cold-start problem: without quality content there are fewer students; without students there is less instructor incentive. Avoid opening with unlimited low-quality uploads.

Use a Founding Instructor Program and initially focus on a few strong categories such as AI/ML, Software Development, Data, Business, Design, Cloud and Cybersecurity. Start with a smaller number of high-quality categories rather than attempting dozens at launch.

35. Mobile Strategy

- First deliver a responsive web experience.
- Add PWA capabilities where useful.
- Consider React Native later when usage demonstrates a need for native apps.

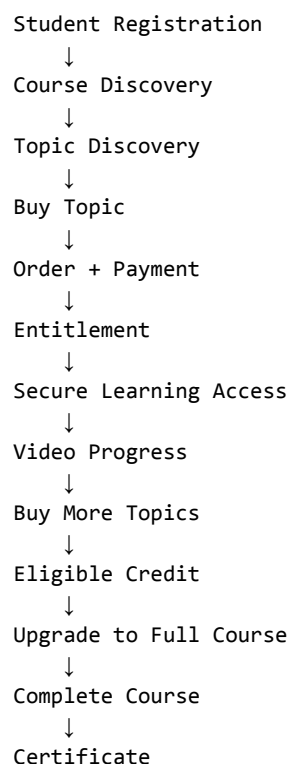
36. Development Phases

Phase	Scope
Phase 1	Foundation: auth, users, roles, profiles, RBAC, admin base, environments, CI/CD
Phase 2	Learning marketplace: courses, modules, topics, lessons, resources, instructor curriculum builder, drafts, publishing
Phase 3	Marketplace: homepage, search, categories, course pages, topic pages, instructor profiles, wishlist, reviews
Phase 4	Commerce: cart, checkout, payments, webhooks, orders, topic purchases, course purchases, credits, upgrades, refunds, entitlements, ledger

Phase 5	Learning: secure video, progress, resources, notes, quizzes, completion, dashboard, certificates
Phase 6	Instructor economics: revenue, commission, balance, payouts, invoices, tax information
Phase 7	Admin: users, courses, topics, orders, refunds, moderation, reports, support
Phase 8	CRM: lead capture, student profile, events, pipeline, campaigns, email and automation
Phase 9	Production: S3, CDN, ALB/ECS, MongoDB Atlas, Redis, monitoring, logging, backups, security, CI/CD
Phase 10	Expansion: live, cohorts, mentorship, community, AI, advanced analytics, internationalization, career and enterprise

37. Recommended First Production Loop

The first production milestone should prove the product's unique value rather than attempting every advanced feature.



This loop should be stable before adding advanced AI, community, complex recommendation ML, custom DRM, native mobile applications or a microservices architecture.

38. Recommended Implementation Order — Starting from the Existing LMS

17. Audit the current React LMS screens and existing data flows.

18. Create the unified authentication and RBAC foundation.
19. Define MongoDB schemas and indexes.
20. Define object-storage naming conventions and file ownership rules.
21. Build the public homepage and marketplace shell.
22. Build student profile and student dashboard.
23. Build instructor profile and instructor dashboard.
24. Build course → module → topic → lesson content builder.
25. Implement instructor upload and S3 asset management.
26. Implement video processing states and adaptive streaming.
27. Implement learning progress and resume playback.
28. Build admin operations dashboard.
29. Build super-admin business dashboard.
30. Build webinar creation, scheduling, registration and recording management.
31. Build course/topic search and discovery.
32. Build cart, checkout and payment integration.
33. Implement orders, entitlements, topic credits and upgrades.
34. Implement refunds and financial ledger.
35. Implement certificates and verification.
36. Implement CRM event tracking.
37. Implement analytics.
38. Add live/community/mentorship features.
39. Add AI features after the underlying data and learning systems are reliable.
40. Only after the core LMS is stable, connect the existing automation/AI workflows as an external automation layer.

39. Storage and Ownership Summary

Asset / Data	Owner / Relation	Primary storage
Student account	User	MongoDB
Instructor account	User + Instructor Profile	MongoDB
Course	Instructor	MongoDB
Module	Course	MongoDB
Topic	Module / Course	MongoDB

Lesson metadata	Topic	MongoDB
Video original	Lesson	S3
Video renditions	Lesson	S3
HLS manifest	Lesson	S3/CDN
Thumbnail	Course/lesson/webinar	S3
PDF/slides/resources	Lesson/topic/webinar	S3
Webinar metadata	Instructor	MongoDB
Webinar recording	Webinar	S3
Order	Student	MongoDB
Payment	Order	MongoDB + payment provider
Entitlement	Student + product	MongoDB
Topic credit	Student + course	MongoDB
Learning progress	Student + lesson	MongoDB
Certificate	Student + course	MongoDB + generated asset storage
CRM events	Student	MongoDB / event infrastructure
Audit log	Actor + resource	MongoDB / log storage

40. What Should Not Be Built First

- AI tutor as the first milestone
- Large-scale live streaming infrastructure
- Native Android/iOS applications
- Microservices from day one
- Custom DRM
- Custom video infrastructure
- Complex ML recommendation system
- Many payment providers before business requirements are known
- Dozens of countries before the payment/tax model is established
- Hundreds of categories before marketplace liquidity exists
- Enterprise SSO
- Advanced mentorship
- Complex gamification

41. Product Demo Journey

The clearest demonstration of the unique product should be a short end-to-end topic-credit journey.

41. Student searches for 'React Authentication'.

42. A complete React + Node.js course is shown.
43. Student expands the Authentication module.
44. Individual topics such as JWT Basics, Access Tokens, Refresh Tokens, OAuth 2.0 and RBAC show separate prices.
45. Student buys Refresh Tokens.
46. Student learns the topic.
47. Student later buys OAuth and RBAC.
48. The platform calculates eligible historical credit.
49. The platform presents the remaining amount to upgrade to the full course.
50. Student upgrades and all entitled topics unlock.
51. Student completes the course and receives a certificate.

42. Long-Term Product Roadmap

PHASE 1 – FOUNDATION

Auth • Users • Roles • Architecture

PHASE 2 – LEARNING MARKETPLACE

Courses • Modules • Topics • Search • Discovery

PHASE 3 – ATOMIC COMMERCE

Topic pricing • Checkout • Entitlements • Credits • Upgrade • Refunds

PHASE 4 – LEARNING EXPERIENCE

Video • Progress • Quiz • Resources • Certificates

PHASE 5 – CREATOR ECONOMY

Instructor • Earnings • Payouts • Analytics • Reviews

PHASE 6 – EDUCATION CRM

Leads • Events • Campaigns • Automation • Engagement

PHASE 7 – GLOBAL SCALE

Multi-currency • Tax • Localization • CDN • AWS • Security

PHASE 8 – LIVE + COMMUNITY

Live • Cohorts • Mentorship • Community

PHASE 9 – AI

AI Tutor • AI Instructor • AI CRM • AI Recommendations

PHASE 10 – CAREER / ENTERPRISE

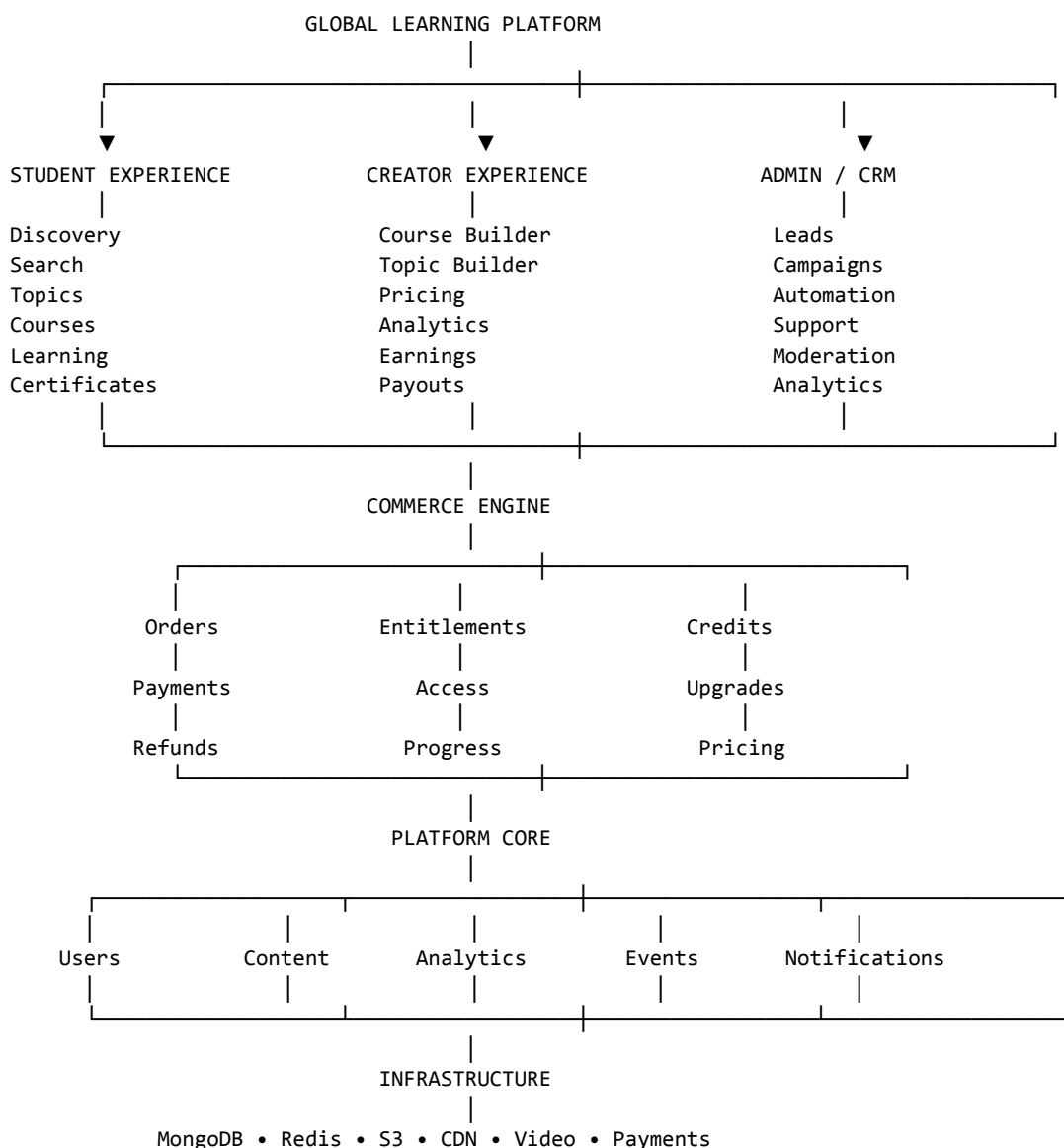
Learning Paths • Skill Graph • Career Profiles • Enterprise LMS • B2B Training

43. Planning Estimates

The supplied specification gives planning estimates rather than guarantees. It describes approximately 20–28 weeks (5–7 months) for a serious production MVP with a capable 3–5 person team, approximately 5–6 months with five developers, and approximately 7–9 months with three developers. A strong marketplace platform is described at roughly 9–12 months and a mature global platform at 12–18+ months. Content operations, integrations, compliance and payment-provider onboarding can materially change these timelines.

These are planning estimates from the supplied specification, not commitments.

44. Final Architecture



45. Final Development Principle

Build the platform as a coherent product, not as a collection of unrelated dashboards. The database, storage, content hierarchy, entitlement system and progress system must be established first. Every dashboard should

consume the same underlying source of truth, while RBAC determines which records and actions each role can access.

The most important business and technical loop remains: Discover a topic → buy only that topic → receive entitlement → learn → buy more → accumulate eligible credit → upgrade to the complete course → complete learning → receive certificate → continue into a learning path/career journey.

This document deliberately treats the existing automation/AI workflow as a later integration layer. The LMS itself must remain independently functional: students should be able to discover, purchase, learn, track progress and receive certificates without depending on an automation workflow.

Yes — you mean **continue the improvement list from Point 45**, including the changes I mentioned previously, rather than only rewriting Point 46.

46. Additional Platform Improvements — HIGH Priority

Add the following improvements to make the LMS more complete and production-ready:

- **Notification Center**
 - Centralized notifications for students, instructors, counselors, and admins.
 - Course updates, enrollment updates, assignments, payments, certificates, messages, etc.
 - Read/unread status and notification history.
- **Content Versioning**
 - Maintain versions of courses, modules, topics, lessons, quizzes, and resources.
 - Track what changed and when.
 - Allow instructors/admins to review previous versions where required.
- **Content Lifecycle Management**
 - Introduce clear content states:
Draft → Review → Published → Archived
 - Prevent incomplete content from being publicly accessible.
 - Admin/instructor approval workflow where required.
- **Advanced Search & Filtering**
 - Search across courses, topics, instructors, learning paths, categories, skills, etc.
 - Filters for price, level, language, rating, duration, format, category, and availability.
 - Prepare architecture for OpenSearch/Algolia later.
- **Assignment & Quiz System**
 - Assignments, quizzes, questions, attempts, scores, deadlines, submissions, and evaluations.
 - Track student performance and completion.
 - Instructor/admin grading functionality.
- **Content Access Rules**
 - Define who can access specific courses, modules, topics, lessons, resources, webinars, and recordings.
 - Support free, paid, purchased, enrolled, subscription, and role-based access.
 - Always enforce access on the backend.
- **Audit & Activity History**
 - Record important actions such as login, course publishing, price changes, refunds, enrollment changes, role changes, content edits, and admin actions.
 - Provide an activity/audit timeline for administrators.
- **Backup & Disaster Recovery**

- Automated database backups.
- Storage backup/versioning for important files.
- Recovery procedures and retention policies.
- Disaster-recovery planning for production.
- **SEO Architecture**
 - SEO-friendly course/topic/instructor pages.
 - Metadata, canonical URLs, structured data, sitemap, robots.txt, and indexable public pages.
 - SEO-friendly slugs.
- **Support & Help Center**
 - FAQ/help articles.
 - Student support requests/tickets.
 - Instructor support.
 - Admin support management.
 - Ticket status and communication history.
- **Coupon & Promotion Engine**
 - Coupons, percentage/fixed discounts, promotional campaigns, bundles, expiry dates, usage limits, and eligibility rules.
 - Integrate promotions with topic purchases and course upgrades.
- **Data Ownership & Deletion**
 - Student account/data export.
 - Account deletion/deactivation.
 - Data-retention rules.
 - Proper handling of associated learning, payment, communication, and certificate records.

47. Unified Communication & Conversation Center — HIGH Priority

Add a centralized **Omnichannel Communication Inbox** instead of managing each communication channel separately.

Channels:

- WhatsApp
- Email
- SMS
- In-app Chat
- Voice/Call history
- Notifications

Core functionality:

- One unified conversation timeline per student.
- All messages/interactions linked to the student profile.
- Counselor can see previous conversations and student activity.
- Reply through the appropriate channel.
- Track inbound/outbound communication.
- Call/callback history.
- Conversation status: **Open / Pending / Resolved**.
- Assign conversations to counselors.
- Search and filter conversations.

- Communication history inside the student profile.
- Connect communication events with course views, enquiries, purchases, applications, and follow-ups.

Goal: Give counselors a **single 360° view of the student's communication and journey** instead of switching between separate communication systems.

Priority: HIGH

Communication Inbox

You currently have communication records, but I would add a **unified inbox**.

Instead of:

Email app

WhatsApp

Telegram

SMS

separately:

OMNICHANNEL INBOX

Student



Counselor sees everything in one conversation timeline.

Meritto currently emphasizes centralized WhatsApp/live conversations and communication history inside the student profile.

Priority: HIGH

EDTECH LMS — REAL-TIME PHASE IMPLEMENTATION DOCUMENTATION

Global Learning Marketplace + Modular Learning Engine + Creator Platform + Education CRM

1. Purpose

This document is an implementation-ready phase plan for building the LMS as one connected product. Every phase defines what was achieved previously, the goal and aim of the current phase, screen/features, important fields, API/data connections, technology, and Definition of Done.

2. Frontend Technology Rule

Project frontend technology: React.js + TypeScript. HTML/CSS are not listed as project technologies.

3. Core Technology Stack

- Frontend: React.js + TypeScript
- Backend: Node.js + Express.js
- Database: MongoDB Atlas
- Authentication: JWT + Refresh Tokens + OAuth
- API: REST under /api/v1
- Search: OpenSearch or Algolia
- Storage: AWS S3
- Video: MediaConvert + HLS + CloudFront
- Payments: CashFree + regional/local provider where required
- Email: AWS SES; later WhatsApp, SMS, Push, Telegram and in-app communication
- Live learning: LiveKit or managed live provider
- Queue/jobs: SQS
- Monitoring: Sentry + CloudWatch
- Deployment: Docker + ECS/Fargate + ALB + Route53 + ACM
- CI/CD: GitHub Actions
- Architecture: modular monolith first; extract services only when justified

4. Standard Screen-to-Backend Connection

React.js + TypeScript Screen → API Client → FastAPI Route → Service/Business Logic → Repository/Data Access → MongoDB/Redis/S3/External Provider → Response DTO → TypeScript State → UI

5. Major Screen/Data Contracts

Screen	Main Fields	API	Primary Data	Access
Homepage	heroSearch, categories, trendingTopics, popularCourses, learningPaths, webinars, topInstructors, freeTopics, newCourses, recommendations, testimonials, footer	GET /api/v1/homepage; category/course/topic /instructor/webinar endpoints	courses, topics, instructors, categories, learning_paths, webinars, reviews	PUBLIC
Student Dashboard	continueLearning, progress, purchasedTopics, upgradeOffers, webinars, certificates, orders, wishlist, notifications, profile	GET /api/v1/dashboard/student; /users/me; /learning/my-courses; /entitlements; /progress; /orders; /certificates; /wishlist; /notifications	users, entitlements, learning_progress, video_progress, orders, certificates, wishlist, notifications, webinars	STUDENT
Instructor Dashboard	profile completion, verification/KYC, course/topic/student counts, rating, reviews, revenue, available/pending balance, payouts, courses, live sessions	GET /api/v1/dashboard/instructor; /instructor/profile; /courses; /students; /earnings; /payouts; /reviews	instructor_profiles, courses, topics, users, reviews, instructor_earnings, payouts, live_sessions	INSTRUCTOR
Admin Dashboard	KPI cards, users, instructors, course/topic approval, webinars, orders, payments, refunds, reviews, support, marketing, reports	GET /api/v1/dashboard/admin; /admin/users; /admin/instructors; /admin/courses; /admin/orders; /admin/refunds; /admin/support	users, instructor_profiles, courses, topics, webinars, orders, payments, refunds, reviews, support, audit_logs	ADMIN
Super Admin	GMV, revenue, takeRate, students, creators, countries, currencies, taxes, growth, fraud, infrastructure	GET /api/v1/dashboard/super-admin	orders, payments, financial ledger, payouts, analytics, countries, currencies, taxes, fraud, infrastructure	SUPER_ADMIN
CRM Dashboard	leads, pipeline, source, campaign, interest, lastActivity, engagement, owner, nextAction	GET /api/v1/crm/dashboard	leads, lead_events, campaigns, users, communication_logs, CRM events	ADMIN/CRM
Communication Inbox	conversation list, channel, student, priority, status, assignee, messages, attachments, timeline, reply, call history, search/filter	GET /api/v1/communications/conversations; /conversations/:id; /messages	communication_logs, users, CRM events, notifications	ADMIN/CRM

6. Real-Time Phase Implementation Plan

1. Foundation & Engineering Setup

Previous phase achieved: No previous phase; establish the base.

Goal: Create repository, environments, coding conventions and modular backend/frontend structure.

Aim / what must be achieved: React.js + TypeScript app, FastAPI app, configuration, API client, error/loading conventions, Docker, CI/CD skeleton.

Important fields / values: environment, apiBaseUrl, appVersion, buildVersion, featureFlags

Technology / stack: GitHub Actions; Docker; FastAPI; React.js + TypeScript; MongoDB Atlas

Expected output / Definition of Done: A clean development environment runs end-to-end and a health API is reachable.

2. Authentication, User Model & RBAC

Previous phase achieved: Foundation is running.

Goal: Create secure identity and role access.

Aim / what must be achieved: Registration/login, refresh tokens, OAuth, verification, password flows, roles STUDENT/INSTRUCTOR/ADMIN/SUPER_ADMIN.

Important fields / values: userId, firstName, lastName, email, phone, passwordHash, profilePhoto, country, state, city, timezone, preferredLanguage, role, status, emailVerified, createdAt, updatedAt, lastLoginAt

Technology / stack: JWT, OAuth, FastAPI, MongoDB

Expected output / Definition of Done: A user can authenticate and only authorized roles can access protected routes.

3. Public Homepage

Previous phase achieved: Authentication foundation exists.

Goal: Build the public discovery entry point and connect real marketplace data.

Aim / what must be achieved: Hero search, categories, trending topics, popular courses, learning paths, webinars, instructors, free topics, new courses, recommendations, testimonials, footer.

Important fields / values: heroTitle, heroSubtitle, searchPlaceholder, categoryId/name, topicId/title/price, courseId/title/price, rating, reviewCount, studentCount, instructorId/name, pathId, webinarId, startTime, isFree, thumbnail, slug

Technology / stack: React.js + TypeScript + FastAPI

Expected output / Definition of Done: Homepage loads from APIs with loading, empty and error states; links resolve to real entities.

4. Student Profile & Student Dashboard

Previous phase achieved: Authentication and homepage exist.

Goal: Give students a connected personal learning workspace.

Aim / what must be achieved: Profile, continue learning, progress, purchased topics, upgrade offers, webinars, certificates, orders, wishlist, notifications.

Important fields / values: firstName, lastName, email, phone, photo, country, state, city, timezone, language, education, skills; courseId, title, progressPercent, lastWatchedAt; topicId, purchaseDate; upgradeCourseId, coursePrice, eligibleCredit, upgradePrice; certificateId, issueDate, verificationUrl; orderId, amount, status

Technology / stack: React.js + TypeScript + FastAPI + MongoDB

Expected output / Definition of Done: Student sees only their own records and can navigate from dashboard cards into learning/commerce screens.

5. Instructor Profile & Instructor Dashboard

Previous phase achieved: Auth and student/public foundations exist.

Goal: Create creator identity, metrics and earnings workspace.

Aim / what must be achieved: Profile, verification/KYC, courses/topics, students, rating/reviews, revenue, balances, payouts, live sessions, quick actions.

Important fields / values: instructorId, bio, expertise[], skills[], experience, qualifications[], verificationStatus, kycStatus, payoutStatus, courseCount, topicCount, studentCount, rating, grossRevenue, availableBalance, pendingBalance

Technology / stack: React.js + TypeScript + FastAPI

Expected output / Definition of Done: Instructor can manage profile and see ownership-scoped metrics.

6. Course Builder & Content Hierarchy

Previous phase achieved: Instructor workspace exists.

Goal: Create Topic → Module → Course → Lesson content model.

Aim / what must be achieved: Course builder, modules, topics, lessons, resources, ordering, pricing, publishing states.

Important fields / values: courseId, title, slug, description, shortDescription, thumbnail, banner, instructorId, category, subcategory, level, language, skills[], requirements[], learningObjectives[], coursePrice, currency, status, visibility, publishedAt; moduleId; topicId; topic price/duration/difficulty/skills/prerequisites/learningObjectives/isFree/status/order; lessonId, videoId, playbackReference, duration, transcript, resources[], order/status

Technology / stack: React.js + TypeScript + FastAPI + MongoDB

Expected output / Definition of Done: Instructor can create a complete draft hierarchy and publish only valid content.

7. Admin Operations Dashboard & Moderation

Previous phase achieved: Courses/content can be created.

Goal: Connect operational governance.

Aim / what must be achieved: KPI cards, users, instructor approvals, course/topic approvals, webinar management, orders, refunds, reviews, support and reports.

Important fields / values: userCount, studentCount, instructorCount, pendingInstructorApprovals, pendingCourses, pendingTopics, ordersToday, refundsPending, openSupportTickets, flaggedReviews

Technology / stack: FastAPI RBAC + audit logging

Expected output / Definition of Done: Admin actions are authorization-protected and auditable.

8. Video, Files & Secure Media

Previous phase achieved: Lessons and resources exist.

Goal: Connect instructor uploads to secure student playback.

Aim / what must be achieved: Upload authorization, S3 objects, processing status, HLS pipeline, CloudFront delivery and signed playback.

Important fields / values: videoId, lessonId, sourceObjectKey, processingStatus, duration, thumbnailObjectKey, hlsManifestObjectKey, availableQualities[], captions, transcript

Technology / stack: S3 + MediaConvert + CloudFront + FastAPI

Expected output / Definition of Done: Videos process to playable assets; permanent unrestricted video URLs are not exposed.

9. Marketplace Search, Course Pages & Topic Pages

Previous phase achieved: Homepage and course content exist.

Goal: Make courses and atomic topics discoverable and commercially visible.

Aim / what must be achieved: Search/filter, course detail, topic detail, instructor detail, ratings, individual topic prices, full-course price, upgrade messaging.

Important fields / values: courseId, topicId, title, description, price, currency, rating, reviewCount, instructorId, skills, prerequisites, learningObjectives, slug

Technology / stack: OpenSearch/Algolia + FastAPI

Expected output / Definition of Done: Search returns courses/topics/instructors/paths and detail pages use live data.

10. Cart, Checkout, Payment, Order & Entitlement

Previous phase achieved: Products are discoverable.

Goal: Create the commercial transaction and access foundation.

Aim / what must be achieved: Cart, checkout, payment provider, verified webhooks, order creation, payment status and entitlements.

Important fields / values: orderId, userId, items[], subtotal, discount, tax, credit, finalAmount, currency, paymentStatus, orderStatus, paymentId

Technology / stack: Stripe Connect + regional provider + FastAPI

Expected output / Definition of Done: Successful payment creates an order and server-side entitlement; failed payments do not grant access.

11. Topic Credit, Course Upgrade, Refund & Ledger

Previous phase achieved: Orders and entitlements exist.

Goal: Implement the platform's atomic commerce model.

Aim / what must be achieved: Eligible topic credit, course upgrade remainder, refunds, entitlement revocation and financial ledger.

Important fields / values: creditId, userId, courseId, sourceTopicPurchases[], eligibleAmount, usedAmount, remainingAmount, status; upgradeId, coursePrice, upgradePrice; ledgerId, type, amount

Technology / stack: FastAPI business rules + MongoDB

Expected output / Definition of Done: Topic purchases can become eligible course credit; refunds/upgrade calculations are auditable.

12. Learning Player, Progress & Resources

Previous phase achieved: Entitlements provide access.

Goal: Build secure learning experience.

Aim / what must be achieved: Course/topic player, video progress, completion, resources, resume position.

Important fields / values: userId, courseId, moduleId, topicId, lessonId, watchedSeconds, totalSeconds, progressPercent, completed, lastWatchedAt

Technology / stack: React.js + TypeScript + FastAPI + secure media

Expected output / Definition of Done: Authorized students can learn and progress is persisted accurately.

13. Quiz & Assignment System

Previous phase achieved: Learning player exists.

Goal: Add assessment and assignment workflows.

Aim / what must be achieved: Quiz authoring, questions, attempts, scoring, pass rules, assignments, submissions and feedback.

Important fields / values: quizId, courseId, topicId, questions[], passingScore, attemptLimit, duration; questionId, type, options[], correctAnswer, marks; attemptId, answers[], score, percentage, passed; assignmentId, deadline, submissionId, feedback

Technology / stack: React.js + TypeScript + FastAPI

Expected output / Definition of Done: Attempts are validated server-side and scores/statuses are stored.

14. Completion, Certificates & Verification

Previous phase achieved: Progress and assessments exist.

Goal: Turn completion into verifiable achievement.

Aim / what must be achieved: Completion rules, certificate issuance, verification page and certificate asset.

Important fields / values: certificateId, certificateNumber, userId, courseId, instructorId, completionPercentage, issueDate, verificationUrl, certificateAssetUrl, status

Technology / stack: FastAPI + storage + React.js + TypeScript

Expected output / Definition of Done: Eligible students receive a unique certificate with a working verification URL.

15. Webinars & Live Learning

Previous phase achieved: Instructor and student accounts exist.

Goal: Connect live/cohort learning.

Aim / what must be achieved: Webinar creation, scheduling, capacity, enrollment, meeting provider, attendance and recordings.

Important fields / values: webinarId, title, description, thumbnail, instructorId, category, startTime, endTime, timezone, capacity, meetingProvider, meetingUrl, recordingUrl, price, currency, paymentRequired, status

Technology / stack: LiveKit/managed live + FastAPI

Expected output / Definition of Done: A learner can discover, enroll and access an authorized live session.

16. Learning Paths

Previous phase achieved: Courses/topics exist.

Goal: Combine atomic learning units into career-oriented paths.

Aim / what must be achieved: Path builder, skills, level, duration, courses/topics and path completion.

Important fields / values: learningPathId, title, slug, description, skills[], career, level, estimatedDuration, items[], courses[], topics[], certificate, status

Technology / stack: React.js + TypeScript + FastAPI

Expected output / Definition of Done: Paths can reference existing courses/topics without duplicating content.

17. CRM & Student 360

Previous phase achieved: Users, purchases and learning events exist.

Goal: Create a unified lead/student record for education operations.

Aim / what must be achieved: Lead capture, pipeline, source/campaign, interest, owner, activity timeline and purchase/learning context.

Important fields / values: leadId, studentId, name, email, phone, country, source, campaign, interest, courses[], topics[], purchases[], learningProgress, lastActivity, engagement, pipelineStage, ownerId

Technology / stack: FastAPI + MongoDB

Expected output / Definition of Done: Counselors/admins can view a unified student journey.

18. Events & Automation

Previous phase achieved: Commerce, learning and CRM data exist.

Goal: Create the event backbone used by analytics and automation.

Aim / what must be achieved: Emit events for views, purchases, upgrades, learning, completion and certificates.

Important fields / values: eventId, eventType, userId, entityType, entityId, metadata, timestamp, source, campaignId

Technology / stack: FastAPI + SQS/worker layer

Expected output / Definition of Done: Business events are consistently recorded and can trigger downstream actions.

19. Unified Communication Inbox

Previous phase achieved: CRM and communication logs exist.

Goal: Create one operational communication workspace.

Aim / what must be achieved: Conversation list, channel, student, priority, assignee, messages, attachments, timeline, reply, call history and filters.

Important fields / values: conversationId, studentId, assignedCounselorId, channel, subject, status, priority, lastMessageAt, createdAt, resolvedAt, messageId, senderId, receiverId, messageType, content, attachments[], direction, deliveryStatus, readAt

Technology / stack: Email/WhatsApp/SMS/Telegram/in-app integrations + FastAPI

Expected output / Definition of Done: Messages remain linked to the correct student record and conversation.

20. Notification Center

Previous phase achieved: Events and communications exist.

Goal: Give users reliable in-product notification visibility.

Aim / what must be achieved: Unread/read state, deep links, event-based notifications and preferences.

Important fields / values: notificationId, userId, type, title, message, entityType, entityId, isRead, createdAt

Technology / stack: FastAPI + React.js + TypeScript

Expected output / Definition of Done: Notifications are generated from supported events and deep-link correctly.

21. Content Versioning, Review & Moderation

Previous phase achieved: Course/content publishing exists.

Goal: Protect quality and create traceable review history.

Aim / what must be achieved: Draft/review/publish lifecycle, snapshots, reviewer notes, moderation and rollback-ready history.

Important fields / values: contentId, version, changedBy, changeSummary, snapshot, status, createdAt, reviewerId, reviewNotes

Technology / stack: MongoDB + FastAPI + audit logging

Expected output / Definition of Done: Every reviewed content change is traceable.

22. Support, Help Center & Tickets

Previous phase achieved: Users and communication exist.

Goal: Create structured customer support.

Aim / what must be achieved: Help articles, ticket creation, priority, assignment, status, attachments and resolution.

Important fields / values: ticketId, userId, category, subject, description, priority, status, assignedTo, attachments[], createdAt, updatedAt, resolvedAt

Technology / stack: React.js + TypeScript + FastAPI

Expected output / Definition of Done: Support cases can be created, assigned, updated and resolved.

23. Marketing, Coupons & Promotions

Previous phase achieved: Commerce is operational.

Goal: Add controlled discount and promotion management.

Aim / what must be achieved: Coupon creation, eligibility, limits, dates, product/user targeting and promotion reporting.

Important fields / values: couponId, code, discountType, discountValue, currency, minimumAmount, maximumDiscount, usageLimit, perUserLimit, startDate, expiryDate, eligibleProducts[], eligibleUsers[], status, promotionId, name

Technology / stack: FastAPI commerce rules + Admin UI

Expected output / Definition of Done: Coupons are validated server-side and financial totals remain consistent.

24. Analytics & Reporting Dashboards

Previous phase achieved: Events, orders and learning data exist.

Goal: Turn operational data into measurable dashboards.

Aim / what must be achieved: Marketplace, learning, instructor, CRM and commercial analytics; topic-to-course conversion.

Important fields / values: metric, dimension, value, date, courseId, topicId, instructorId, country, channel, campaignId

Technology / stack: MongoDB aggregation initially; scalable analytics later

Expected output / Definition of Done: Reports match source records and respect role access.

25. SEO & Public Discoverability

Previous phase achieved: Public marketplace pages exist.

Goal: Make public courses/topics/instructors discoverable.

Aim / what must be achieved: SEO metadata, canonical URLs, slugs and structured data for public pages.

Important fields / values: seoTitle, seoDescription, canonicalUrl, slug, structuredData, indexable

Technology / stack: React.js + TypeScript + FastAPI

Expected output / Definition of Done: Public entities have stable URLs and correct metadata.

26. Security, Audit & Access Governance

Previous phase achieved: Core product workflows exist.

Goal: Harden the platform and make sensitive actions traceable.

Aim / what must be achieved: RBAC, ownership checks, audit events, payment webhook verification, secure media authorization, rate limits and security testing.

Important fields / values: auditId, actorId, action, resourceType, resourceId, oldValue, newValue, ipAddress, timestamp

Technology / stack: JWT/RBAC + FastAPI + Sentry/CloudWatch

Expected output / Definition of Done: Unauthorized access and sensitive operations are blocked/logged.

27. Backup & Disaster Recovery

Previous phase achieved: Production data is accumulating.

Goal: Protect data and validate recovery.

Aim / what must be achieved: Database backups, object-storage policies, retention and restore tests.

Important fields / values: backupId, resourceType, resourceId, backupDate, retentionDate, status, restoreTestDate

Technology / stack: MongoDB Atlas backups + S3 lifecycle + documented recovery process

Expected output / Definition of Done: A restore procedure is tested, not merely configured.

28. Globalization

Previous phase achieved: Core marketplace and commerce work.

Goal: Support international learners and creators.

Aim / what must be achieved: Locales, currencies, timezone handling, subtitles/audio/transcripts and country-aware pricing/tax integration.

Important fields / values: language, locale, currency, price, displayPrice, exchangeRate, timezone, subtitleLanguage, audioLanguage, transcriptLanguage

Technology / stack: React.js + TypeScript i18n + FastAPI

Expected output / Definition of Done: Prices and time-based events display correctly for supported locales.

29. Community & Mentorship

Previous phase achieved: Learning and live learning exist.

Goal: Add interaction around learning.

Aim / what must be achieved: Course/topic discussions, study groups, announcements, mentor availability, bookings and sessions.

Important fields / values: communityId, courseId, topicId, userId, postId, content, parentId, mentorId, availability, bookingId, sessionId, status

Technology / stack: React.js + TypeScript + FastAPI

Expected output / Definition of Done: Authorized learners can participate and mentors can manage availability/bookings.

30. Recommendation Engine

Previous phase achieved: Behavior/events/analytics exist.

Goal: Personalize discovery without blocking the core product.

Aim / what must be achieved: Recommended courses/topics/paths, reasons, scoring and model versioning.

Important fields / values: recommendationId, userId, sourceEntity, recommendedEntity, reason, score, modelVersion, createdAt

Technology / stack: Rules/analytics first; ML later

Expected output / Definition of Done: Recommendations are explainable and can be disabled without breaking marketplace flows.

31. AI Layer

Previous phase achieved: Learning, CRM and event context exist.

Goal: Add AI only after reliable source data exists.

Aim / what must be achieved: Student tutor, learning assistant, instructor assistant and CRM AI; retrieval-grounded responses.

Important fields / values: assistantSessionId, userId, contextEntityIds[], question, retrievedSources[], response, feedback, model, modelVersion, createdAt

Technology / stack: LLM provider + retrieval layer + FastAPI

Expected output / Definition of Done: AI responses are scoped to authorized data and do not become the source of truth for entitlements/payments.

32. Mobile / PWA

Previous phase achieved: Desktop web workflows are stable.

Goal: Provide mobile-friendly product access without changing business logic.

Aim / what must be achieved: Responsive student/instructor/admin flows, installable PWA capabilities where appropriate, push token/device tracking.

Important fields / values: device, appVersion, pushToken, lastActiveAt

Technology / stack: React.js + TypeScript + PWA approach

Expected output / Definition of Done: Core workflows work on supported mobile screens using the same APIs.

33. Production Infrastructure & Scale

Previous phase achieved: Feature-complete product is validated.

Goal: Prepare stable production operation and scaling.

Aim / what must be achieved: Production environments, observability, queues, CDN, autoscaling, domains, certificates, release process and capacity planning.

Important fields / values: deploymentId, releaseVersion, environment, healthStatus, createdAt, rollbackVersion

Technology / stack: Docker + ECS/Fargate + ALB + Route53 + ACM + SQS + CloudWatch

Expected output / Definition of Done: Production deployment is repeatable and monitored with rollback capability.

34. Full QA, E2E Validation & Launch

Previous phase achieved: All required production workflows are implemented.

Goal: Validate the entire connected system before launch.

Aim / what must be achieved: Unit, integration, E2E, security, payment, entitlement, refund, upgrade, media, role and regression testing.

Important fields / values: testCaseId, suite, environment, status, severity, executedAt, defectId, releaseVersion

Technology / stack: Automated testing + CI/CD + Sentry

Expected output / Definition of Done: The complete production loop passes: registration → browse/search → topic purchase → payment → entitlement → learning → progress → more topics → eligible credit → course upgrade → completion → certificate.

7. Key Real-Time Data Flow Examples

Student buys a topic: React Topic Page → POST /api/v1/checkout → FastAPI checkout → payment provider → verified webhook → order → payment → entitlement → credit/ledger/event updates → Student Dashboard + Notification + CRM + Analytics.

Instructor uploads a video: React Course Builder → upload authorization API → S3 → processing job → MediaConvert → HLS → CloudFront → MongoDB metadata/status → instructor sees Ready → entitled student requests signed playback → player → progress API.

Dashboard loading: React dashboard → dashboard endpoint → backend aggregates authorized records → typed response DTO → TypeScript state → cards/tables/charts.

8. Dashboard-to-Database Connection Matrix

Screen	Endpoint	Primary Data	Access
Homepage	GET /api/v1/homepage	courses, topics, instructors, paths, webinars, categories, reviews	PUBLIC

Student Dashboard	GET /api/v1/dashboard/student	users, entitlements, learning_progress, video_progress, orders, certificates, wishlist, notifications, webinars	STUDENT
Instructor Dashboard	GET /api/v1/dashboard/instructor	instructor_profiles, courses, topics, users, reviews, instructor_earnings, payouts, live_sessions	INSTRUCTOR
Admin Dashboard	GET /api/v1/dashboard/admin	users, instructor_profiles, courses, topics, webinars, orders, payments, refunds, reviews, support, audit_logs	ADMIN
Super Admin Dashboard	GET /api/v1/dashboard/super-admin	orders, payments, financial ledger, payouts, analytics, countries, currencies, taxes, fraud, infrastructure	SUPER_ADMIN
CRM Dashboard	GET /api/v1/crm/dashboard	leads, lead_events, campaigns, users, communication_logs, CRM events	ADMIN/CRM
Communication Inbox	GET /api/v1/communications/conversations	communication_logs, users, CRM events, notifications	ADMIN/CRM
Analytics Dashboard	GET /api/v1/analytics/*	event aggregates, orders, learning_progress, instructor_earnings, CRM events	ADMIN/SUPER_ADMIN

9. No Screen Without a Data Contract

1. Define UI fields before coding.
2. Map every UI field to a database field or computed value.
3. Define the FastAPI endpoint and request/response contract.
4. Define the matching TypeScript request/response types.
5. Define role authorization and resource ownership.
6. Define loading, empty, error and success states.
7. Connect required business events.
8. Add API/integration/E2E tests.
9. Mark the phase complete only when UI + API + business logic + data/provider + authorization + validation + tests are connected.

10. First Production Loop

Registration → Homepage/Search → Course/Topic Page → Buy Topic → Payment → Order → Entitlement → Secure Learning → Progress → Buy More Topics → Eligible Credit → Course Upgrade → Complete → Certificate.

11. Important Architecture Rules

- Entitlement is the source of truth for access. Never trust frontend state for paid access.
- Payment webhooks must be verified server-side before access is granted.
- Topic-credit and upgrade calculations must be server-side and auditable.
- Instructor ownership must be checked on every instructor-managed resource.
- Do not expose permanent unrestricted video URLs.
- Use a modular monolith first. Extract services later when scale or team boundaries justify it.
- n8n/automation should be an automation layer around the core platform, not the source of truth for users, payments, entitlements or learning progress.
- Advanced AI, complex recommendation ML, native mobile apps and custom DRM should not block the first production loop.

12. Core Data Domains

Identity & access: users, instructor_profiles, roles, sessions, audit_logs

Learning content: courses, modules, topics, lessons, resources, quizzes, assignments

Commerce: orders, order_items, payments, refunds, entitlements, topic_credits, course_upgrades, coupons, promotions

Learning: learning_progress, video_progress, quiz_attempts, certificates

Creator economy: instructor_earnings, payouts

Live/community: webinars, live_sessions, communities, bookings, mentorship

CRM: leads, lead_events, campaigns, communication_logs

Platform: events, notifications, analytics, recommendations, AI sessions

13. Launch Success Criteria

- The public homepage and marketplace load from real APIs.
- Students can register, browse, buy a topic, receive entitlement and learn.
- Students can buy additional eligible topics and see correct course-credit/upgrade pricing.
- Refunds revoke access and update financial records correctly.
- Instructors can create/publish content and see earnings.
- Admins can approve/moderate users and content and inspect transactions.
- Certificates can be issued and publicly verified.
- CRM records connect leads/students to purchases, learning activity and communication.
- All critical actions are role-protected, logged and covered by automated tests.

EDTECH LMS — FINAL MERN BACKEND IMPLEMENTATION PLAN

1. Final Technology Stack

Frontend

React.js + TypeScript

Backend

Node.js + Express.js

ODM

Mongoose

Database

MongoDB Atlas

Authentication

JWT + Refresh Tokens + OAuth

File Storage

AWS S3

Video Processing

AWS MediaConvert

Video Delivery

CloudFront + HLS

Payments

Stripe Connect + Regional Payment Provider

Search

OpenSearch / Algolia

Queue / Background Jobs

AWS SQS / BullMQ

Email

Transactional Email Provider / AWS SES

Live Learning

LiveKit / Managed Provider

Monitoring

Sentry + CloudWatch

Deployment

Docker + AWS ECS/Fargate

CI/CD

GitHub Actions

The infrastructure choices are based on the architecture described in the PDF, with **Node.js/Express** replacing **FastAPI** for the MERN implementation.

2. Backend Architecture

Use a **modular monolith**, not microservices initially.

backend/

|

|— src/

| |— config/

| |— middleware/

| |— utils/

| |— constants/

|

| |— modules/

- | | | — auth/
- | | | — users/
- | | | — instructors/
- | | | — courses/
- | | | — modules/
- | | | — topics/
- | | | — lessons/
- | | | — resources/
- | | | — videos/
- | | | — quizzes/
- | | | — assignments/
- | | | — learning/
- | | | — commerce/
- | | | — payments/
- | | | — entitlements/
- | | | — credits/
- | | | — upgrades/
- | | | — refunds/
- | | | — certificates/
- | | | — webinars/
- | | | — learningPaths/
- | | | — reviews/
- | | | — wishlist/
- | | | — crm/
- | | | — communications/
- | | | — notifications/
- | | | — support/
- | | | — marketing/
- | | | — analytics/

```
| | |— recommendations/
| | |— admin/
| | |— audit/
| |
| |— routes/
| |— app.js
| |— server.js
|
|— tests/
|— package.json
|— .env
```

Every module should follow:

Route

↓

Middleware

↓

Controller

↓

Service

↓

Mongoose Model

↓

MongoDB

Business logic should **not** be placed directly inside controllers.

3. MongoDB Database Design

MongoDB uses **collections**, not SQL tables.

The core collections identified in the PDF are:

users

instructor_profiles

courses

modules

topics

lessons

resources

orders

order_items

payments

refunds

entitlements

topic_credits

course_upgrades

learning_progress

video_progress

quiz_attempts

reviews

ratings

questions

answers

wishlists

cart

certificates

live_sessions

bookings

leads

lead_events

campaigns
communication_logs
notifications
coupons
promotions
payouts
instructor_earnings
audit_logs

4. users

Fields

_id
firstName
lastName
email
phone
passwordHash
profilePhoto

country
state
city
timezone

preferredLanguage
learningPreferences

qualification

institution

graduationYear

skills[]

interests[]

role

status

emailVerified

lastLoginAt

createdAt

updatedAt

Role

STUDENT

INSTRUCTOR

ADMIN

SUPER_ADMIN

The four-role RBAC model and user/profile requirements are explicitly specified in the document.

Mongoose concept

role: {

 type: String,

 enum: ['STUDENT', 'INSTRUCTOR', 'ADMIN', 'SUPER_ADMIN'],

 required: true

}

5. instructor_profiles

_id

userId

bio

expertise[]

skills[]

experience

qualifications[]

currentOrganization

country

identityStatus

verificationStatus

kycStatus

payoutStatus

paymentAccount

providerOnboardingState

createdAt

updatedAt

The PDF separates the instructor profile from the common user account.

6. courses

_id

courseId

title

slug

description

shortDescription

thumbnail

banner

instructorId

category

subcategory

level

language

skills[]

requirements[]

learningObjectives[]

coursePrice

currency

status

visibility

publishedAt

createdAt

updatedAt

Course hierarchy

Course



Module



Topic



Lesson



Resource

These course fields and hierarchy are explicitly defined in the PDF.

7. modules

_id

moduleId

courseId

title

description

order

status

createdAt

updatedAt

8. topics

_id

topicId

moduleId

courseId

title

description

price

currency

duration

difficulty

skills[]

prerequisites[]

learningObjectives[]

isFree

status

order

createdAt

updatedAt

The topic is the central atomic-learning product in the system.

9. lessons

_id

lessonId

topicId

title

videoId

playbackReference

duration

transcript

resources[]

order

status

createdAt

updatedAt

10. resources

_id

courseId

moduleId

topicId

lessonId

name

type

objectKey

mimeType

size

isDownloadable

status

uploadedBy

createdAt

updatedAt

Resources should be stored in S3/object storage; MongoDB stores metadata/reference information.

11. videos

_id

lessonId

videoId

sourceObjectKey

processingStatus

duration

thumbnailObjectKey

hlsManifestObjectKey

availableQualities[]

captions

transcript

createdAt

updatedAt

Processing states

UPLOADING

PROCESSING

READY

FAILED

The PDF specifically defines these video metadata requirements and the S3 → processing → HLS → CDN architecture.

12. quizzes

_id

quizId

courseId

topicId

title

description

questions[]

passingScore

attemptLimit

duration

status

createdBy

createdAt

updatedAt

13. questions

_id

questionId

quizId

questionText

type

options[]

correctAnswer

marks

explanation

order

14. quiz_attempts

_id

attemptId

quizId

userId

answers[]

score

percentage

passed

attemptNumber

startedAt

submittedAt

15. assignments

_id

assignmentId

courseId

topicId

lessonId

title

description

instructions

deadline

maxMarks

status

createdBy

createdAt

updatedAt

16. assignment_submissions

_id

submissionId

assignmentId

userId

content

files[]

submittedAt

status

marks

feedback

gradedBy

gradedAt

The PDF requires assignments, questions, attempts, scores, deadlines, submissions and evaluations.

17. cart

_id

userId

items[]

items[].productType

items[].productId

items[].price

items[].currency

subtotal

discount

tax

credit

finalAmount

createdAt

updatedAt

18. orders

_id

orderId

userId

items[]

subtotal

discount

tax

credit

finalAmount

currency

paymentStatus

orderStatus

paymentId

createdAt

updatedAt

These fields are explicitly identified in the implementation plan.

19. order_items

_id

orderId

productType

productId

courseId

topicId

instructorId

quantity

unitPrice

discount

tax

finalPrice

currency

Product types:

TOPIC

COURSE

WEBINAR

LEARNING_PATH

20. payments

_id

orderId

userId

provider

providerPaymentId

amount

currency

status

paymentMethod

transactionReference

webhookEventId

paidAt

failureReason

createdAt

updatedAt

Payment success must be verified server-side before creating access entitlement. Payment webhook signatures must also be verified.

21. entitlements

This is **extremely important**.

_id

userId

productType

productId

courseId

topicId

orderId

source

status

startsAt

expiresAt

grantedAt

revokedAt

createdAt

updatedAt

Product type

COURSE

TOPIC

Status

ACTIVE

REVOKED

EXPIRED

The PDF explicitly states that entitlement is the authoritative source for determining what a student can access.

22. topic_credits

_id

creditId

userId

courseId

sourceTopicPurchases[]

eligibleAmount

usedAmount

remainingAmount

status

createdAt

updatedAt

This supports:

Buy Topic A



Buy Topic B



Buy Topic C



Calculate eligible credit



Upgrade Course

The PDF specifically identifies these credit fields.

23. course_upgrades

_id

upgradeId

userId

courseId

coursePrice

eligibleCredit

upgradePrice

currency

orderId

status

createdAt

completedAt

24. refunds

_id

orderId

paymentId

userId

amount

currency

reason

status

providerRefundId

processedBy

processedAt

createdAt

updatedAt

Refund flow:

Purchase

↓

Refund Eligibility

↓

Refund

↓

Entitlement Adjustment

↓

Credit Adjustment

↓

Instructor Balance Adjustment

↓

Ledger Update

This is the specified commerce flow.

25. financial_ledger

_id

ledgerId

userId

orderId

type

referenceId

amount

currency

direction

description

balanceBefore

balanceAfter

createdAt

Transaction types:

ORDER_PAYMENT

PLATFORM_FEE

INSTRUCTOR_EARNING

TAX

REFUND

CHARGEBACK

PAYOUT

ADJUSTMENT

COURSE_CREDIT

The PDF explicitly says financial history should use a proper ledger rather than reconstructing it from orders.

26. learning_progress

_id

userId

courseId

moduleId

topicId

lessonId

watchedSeconds

totalSeconds

progressPercent

completed

lastWatchedAt

createdAt

updatedAt

This directly follows the PDF's progress record.

27. video_progress

_id

userId

lessonId

watchedSeconds

totalSeconds

progressPercent

completed

lastPosition

lastWatchedAt

createdAt

updatedAt

This allows the student to resume the video from the previous playback position.

28. certificates

_id

certificateId

certificateNumber

userId

courseId

instructorId

completionPercentage

issueDate

verificationUrl

certificateAssetUrl

status

createdAt

The certificate must have online verification rather than existing only as a downloadable PDF.

29. reviews

_id

userId

courseId

topicId

rating

title

comment

status

isVerifiedPurchase

createdAt

updatedAt

30. ratings

_id

userId

courseId

topicId

rating

createdAt

updatedAt

31. questions / answers

For course/topic Q&A:

questions

_id

userId

courseId

topicId

lessonId

question

status

createdAt

updatedAt

answers

_id

questionId

userId

answer

isAccepted

createdAt

updatedAt

32. wishlists

_id

userId

productType

productId

createdAt

33. webinars

_id

webinarId

title

slug

description

thumbnail

instructorId

category

startTime

endTime

timezone

capacity

meetingProvider

meetingUrl

recordingUrl

price

currency

paymentRequired

status

registrations

attendance

questions

reviews

slides

PDFs

recordings

supportingFiles

createdAt

updatedAt

The PDF explicitly defines webinar basic, schedule, delivery, commerce, status, engagement and resource fields.

34. live_sessions

_id

webinarId

instructorId

provider

providerSessionId

startTime

endTime

status

accessReference

createdAt

updatedAt

35. bookings

_id

userId

liveSessionId

status

bookedAt

cancelledAt

createdAt

updatedAt

36. learning_paths

_id

learningPathId

title

slug

description

skills[]

career

level

estimatedDuration

items[]

courses[]

topics[]

certificate

status

createdBy

createdAt

updatedAt

The PDF explicitly allows learning paths to contain topics from different courses.

37. leads

_id

leadId

studentId

name

email

phone

country

source

campaign

interest

courses[]

topics[]

purchases[]

learningProgress

lastActivity

engagement

pipelineStage

ownerId

createdAt

updatedAt

Pipeline:

VISITOR

↓

LEAD

↓

REGISTERED

↓

TOPIC_INTEREST

↓

TOPIC_BUYER

↓

MULTI_TOPIC_BUYER

↓

COURSE_UPGRADE

↓

COMPLETED

↓

RETURNING_STUDENT

These fields and stages come from the CRM specification.

38. lead_events

_id

leadId

eventType

entityType

entityId

metadata

createdAt

39. campaigns

_id

name

type

channel

targetAudience

startDate

endDate

status

budget

createdBy

createdAt

updatedAt

40. events

The PDF's event architecture should be implemented as a collection even though the earlier core-collection list calls it **lead_events**/event infrastructure.

_id

eventId

eventType

userId

entityType

entityId

metadata

timestamp

source

campaignId

Examples:

COURSE_VIEWED

TOPIC_VIEWED

TOPIC_PREVIEW_STARTED

TOPIC_PREVIEW_COMPLETED

TOPIC_ADDED_TO_CART

CHECKOUT_STARTED

PAYMENT_FAILED

PAYMENT_SUCCESS

TOPIC_PURCHASED

COURSE_PURCHASED

COURSE_UPGRADED

VIDEO_STARTED

VIDEO_COMPLETED

TOPIC_COMPLETED

QUIZ_COMPLETED

COURSE_COMPLETED

41. Unified Communication

conversations

_id

conversationId

studentId

assignedCounselorId

channel

subject

status

priority

lastMessageAt

createdAt

resolvedAt

Channels

WHATSAPP

EMAIL

SMS

TELEGRAM

IN_APP

VOICE

messages

_id

messageId

conversationId

senderId

receiverId

messageType

content

attachments[]

direction

deliveryStatus

readAt

createdAt

These are explicitly specified in the communication implementation section.

42. communication_logs

_id

userId

conversationId

channel

direction

eventType

messageId

status

metadata

createdAt

This connects communication with the CRM/student timeline.

43. notifications

_id

notificationId

userId

type

title

message

entityType

entityId

isRead

readAt

createdAt

The PDF specifically gives these notification fields.

44. notification_preferences

_id

userId

email

whatsapp

sms

push

inApp

courseUpdates

paymentUpdates

marketing

webinarReminders

createdAt

updatedAt

45. Content Versioning

content_versions

_id

contentId

contentType

version

changedBy

changeSummary

snapshot

status

createdAt

reviewerId

reviewNotes

This directly implements the document's content versioning/review requirement.

46. Content Access Rules

content_access_rules

_id

contentType

contentId

accessType

roles[]

requiresPurchase

requiresEnrollment

requiresEntitlement

startDate

endDate

status

createdAt

updatedAt

Access types should support the PDF's:

FREE

PAID

PURCHASED

ENROLLED

SUBSCRIPTION

ROLE_BASED

Backend authorization remains mandatory.

47. Support

support_tickets

_id

ticketId

userId

category

subject

description

priority

status

assignedTo

attachments[]

createdAt

updatedAt

resolvedAt

support_messages

_id

ticketId

senderId

message

attachments[]

createdAt

The PDF specifies ticket creation, priority, assignment, status, attachments and resolution.

48. coupons

_id

couponId

code

discountType

discountValue

currency

minimumAmount

maximumDiscount

usageLimit

usedCount

perUserLimit

startDate

expiryDate

eligibleProducts[]

eligibleUsers[]

status

promotionId

createdAt

updatedAt

49. promotions

_id

promotionId

name

description

type

products[]

discountRules

startDate

endDate

status

createdBy

createdAt

updatedAt

The PDF specifically requires percentage/fixed discounts, bundles, expiry, usage limits and eligibility rules.

50. instructor_earnings

_id

instructorId

orderId

orderItemId

grossAmount

discountAmount

refundAmount

paymentFee

platformCommission

taxAmount

netEarning

currency

status

createdAt

51. payouts

_id

instructorId

amount

currency

paymentProvider

providerPayoutId

status

requestedAt

processedAt

failureReason

createdAt

updatedAt

52. audit_logs

_id

actorId

action

resourceType

resourceId

oldValue

newValue

ipAddress

userAgent

timestamp

Audit actions include:

LOGIN

COURSE_PUBLISHED

COURSE_UPDATED

COURSE_PRICE_CHANGED

REFUND_CREATED

ROLE_CHANGED

ENTITLEMENT_REVOKED

INSTRUCTOR_APPROVED

ADMIN_ACTION

CONTENT_UPDATED

The specification requires sensitive administrative operations to be audited.

53. recommendations

For the later recommendation engine:

_id

userId

sourceEntityType

sourceEntityId

recommendedEntityType

recommendedEntityId

reason

score

modelVersion

createdAt

expiresAt

Recommendation implementation should progress from rules → collaborative filtering → ML → AI personalization.

54. analytics

_id

metric

dimension

value

date

courseId

topicId

instructorId

country

channel

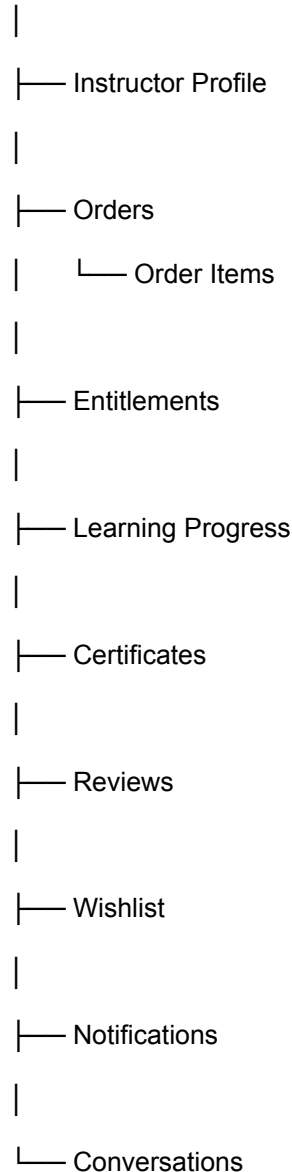
campaignId

createdAt

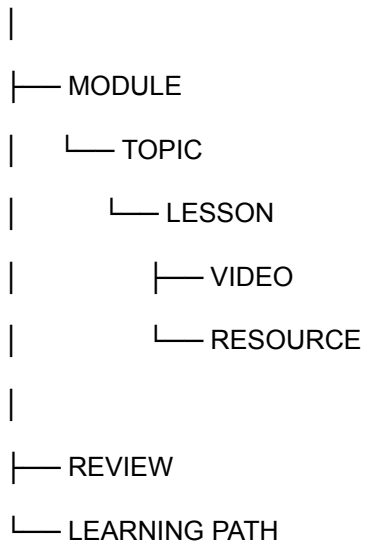
This supports marketplace, learning, instructor, CRM and commercial reporting.

55. Important MongoDB Relationships

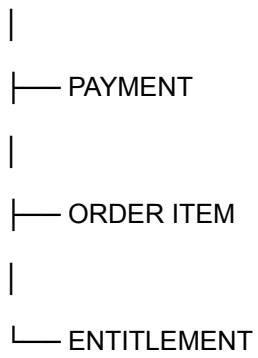
USER



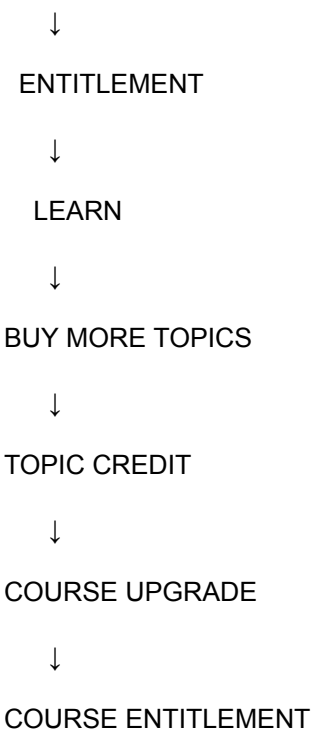
COURSE



ORDER



TOPIC PURCHASE





COURSE COMPLETION



CERTIFICATE

This is the central business loop defined in the PDF.

56. API Structure

Use:

/api/v1/auth

/api/v1/users

/api/v1/instructors

/api/v1/courses

/api/v1/modules

/api/v1/topics

/api/v1/lessons

/api/v1/resources

/api/v1/videos

/api/v1/quizzes

/api/v1/assignments

/api/v1/cart

/api/v1/orders

/api/v1/payments

/api/v1/entitlements

/api/v1/credits

/api/v1/upgrades

/api/v1/refunds

/api/v1/learning

/api/v1/progress

/api/v1/certificates

/api/v1/webinars

/api/v1/live

/api/v1/learning-paths

/api/v1/reviews

/api/v1/wishlist

/api/v1/crm

/api/v1/leads

/api/v1/events

/api/v1/campaigns

/api/v1/communications

/api/v1/notifications

/api/v1/support

/api/v1/coupons

/api/v1/promotions

/api/v1/instructor/earnings

/api/v1/instructor/payouts

/api/v1/analytics

/api/v1/recommendations

/api/v1/admin

/api/v1/super-admin

/api/v1/audit

The original specification also uses `/api/v1` and separates the same major business domains.

57. Authentication & Authorization

Authentication

POST /api/v1/auth/register

POST /api/v1/auth/login

POST /api/v1/auth/refresh

POST /api/v1/auth/logout

POST /api/v1/auth/forgot-password

POST /api/v1/auth/reset-password

POST /api/v1/auth/verify-email

RBAC

STUDENT

INSTRUCTOR

ADMIN

SUPER_ADMIN

Backend rule

Do **not** rely on React route protection.

Frontend Guard

+

Backend RBAC

+

Resource Ownership Check

For example:

Instructor A

↓

PUT /courses/123

↓

Does course 123 belong to Instructor A?

↓

YES → update

NO → 403

The PDF explicitly states that backend authorization is mandatory and instructors must only modify content they own or are authorized to manage.

58. Critical Backend Business Rules

These should be implemented in **services**, not React.

Payment

Payment Success

↓

Verify Provider

↓

Create Order

↓

Create Entitlement

↓

Record Ledger



Emit Event

Topic access

Course owned?

↓ YES

ALLOW

↓ NO

Topic owned?

↓ YES

ALLOW

↓ NO

Topic free?

↓ YES

ALLOW

↓ NO

403

Course upgrade

Existing eligible topic purchases



Calculate credit



Course price



- credit



Upgrade price



Payment



Course entitlement

Course completion

Required lessons completed



Required topics completed



Course completed



Certificate issued



Review request



Next-course recommendation

These flows are directly supported by the PDF's entitlement, commerce, progress and completion architecture.

59. Database Indexes

For the Mongoose implementation, the backend team should create indexes around the documented search/access patterns.

users

email → unique

role

status

courses

slug → unique

instructorId

category

status

visibility

modules

courseId

courseId + order

topics

moduleId

courseId

slug

status

moduleId + order

lessons

topicId

topicId + order

orders

userId

paymentStatus

orderStatus

createdAt

entitlements

userId

productType + productId

userId + courseId

userId + topicId

status

learning_progress

userId + lessonId

userId + courseId

notifications

userId + isRead

userId + createdAt

events

userId + timestamp

eventType + timestamp

entityType + entityId

60. Storage Separation

Do **not** put large videos/PDFs into MongoDB.

MongoDB

|

├─ User metadata

├─ Course metadata

├─ Topic metadata

├─ Lesson metadata

├─ Order/payment metadata

├─ Entitlements

├─ Progress

├─ Quiz data

├─ Certificates metadata

├─ CRM

├─ Events

└─ Notifications

AWS S3

|

└─ Original videos

└─ HLS renditions

└─ Thumbnails

└─ PDFs

└─ Slides

└─ Course resources

└─ Webinar recordings

└─ Certificate assets

CloudFront

|

└─ Secure video/file delivery

Redis

|

└─ Cache / transient data

This storage separation is explicitly specified in the PDF.

61. Final Development Order

The backend team should implement in this order:

Phase 1 — Foundation

Node.js

Express

MongoDB

Mongoose

Environment

Error handling

Validation

Logging

Phase 2 — Authentication

Users

JWT

Refresh tokens

Email verification

Password reset

RBAC

Phase 3 — Content

Courses

Modules

Topics

Lessons

Resources

Videos

Phase 4 — Marketplace

Categories

Search

Course pages

Topic pages

Instructor profiles

Reviews

Wishlist

Learning paths

Phase 5 — Commerce

Cart

Orders

Order Items

Payments

Entitlements

Phase 6 — Atomic Commerce

Topic Credits

Course Upgrades

Refunds

Financial Ledger

Phase 7 — Learning

Learning Progress

Video Progress

Quizzes

Quiz Attempts

Assignments

Submissions

Certificates

Phase 8 — Instructor Economics

Instructor Earnings

Payouts

Phase 9 — Live Learning

Webinars

Live Sessions

Bookings

Attendance

Recordings

Phase 10 — CRM

Leads

Lead Events

Campaigns

Events

Phase 11 — Communication

Conversations

Messages

Communication Logs

Notifications

Phase 12 — Operations

Support

Coupons

Promotions

Content Versioning

Access Rules

Audit Logs

Phase 13 — Analytics & Advanced

Analytics

Recommendations

AI

Community

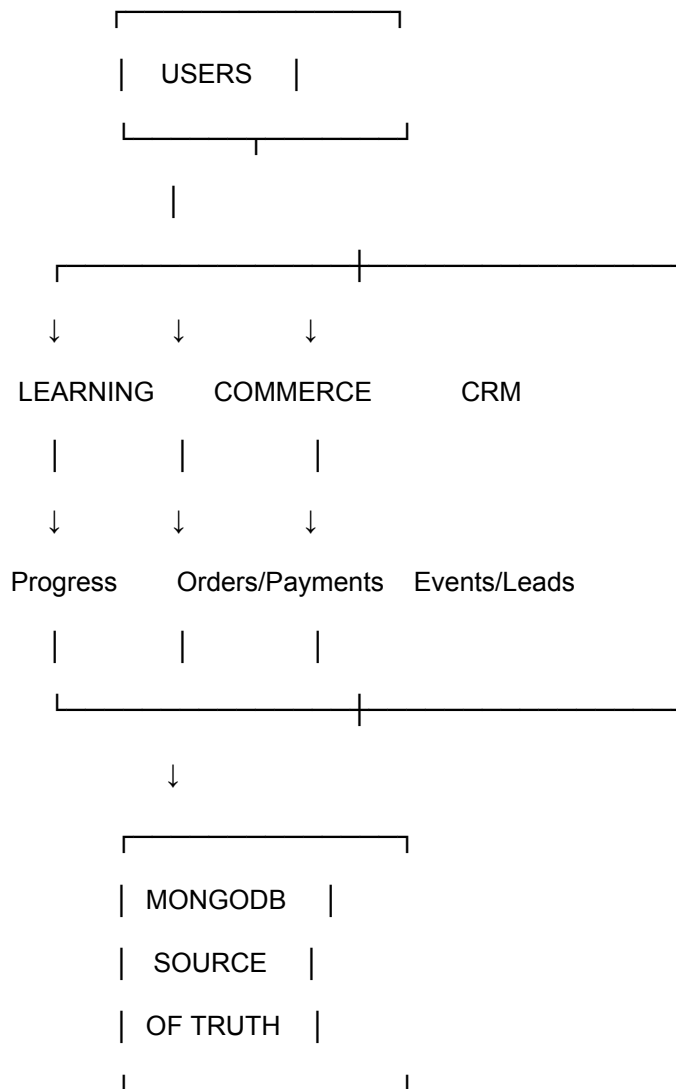
Mentorship

Advanced automation

This follows the implementation order in the source document: foundation → marketplace → content → commerce → learning → instructor economics → admin → CRM → analytics → later expansion.

Most important implementation principle

The backend should be built around **one shared source of truth**, not separate databases for Student, Instructor, Admin and CRM.



This is consistent with the PDF's final principle: the platform should operate as **one coherent product**, with the database, content hierarchy, entitlement system and progress system established first, while RBAC controls what each role can access.

One final note for the backend team: the PDF's original backend technology is **FastAPI + Python**, so the document above is a **MERN implementation translation**, not a claim that the PDF itself specified Node.js/Express. The MongoDB data model and business requirements are retained from the PDF; Node.js, Express and Mongoose are the implementation choices for your requested MERN stack.